

Getting Started with Python Testing Using Pytest

Pytest로 시작하는 파이썬 테스트

테스트의 개념과 pytest 시작하기

테스트

프로젝트가 커질수록 기능 추가나 구조 변경 시 기존 기능이 깨지는 일이 빈번하게 발생함

테스트 없는 코드 수정은 예기치 않은 오류를 반드시 동반함



이 부분만 바꿨는데
왜 다른 기능이 안 되지?

테스트란?



어떤 입력을 넣었을 때, 기대한 결과가 정확히 나오는지 **코드로 검증하는 과정**



단위 테스트(Unit Test)



- 개별 코드 단위(메서드, 클래스) 검증
- 격리된 환경에서 빠르게 실행
- 회귀 테스트의 기반

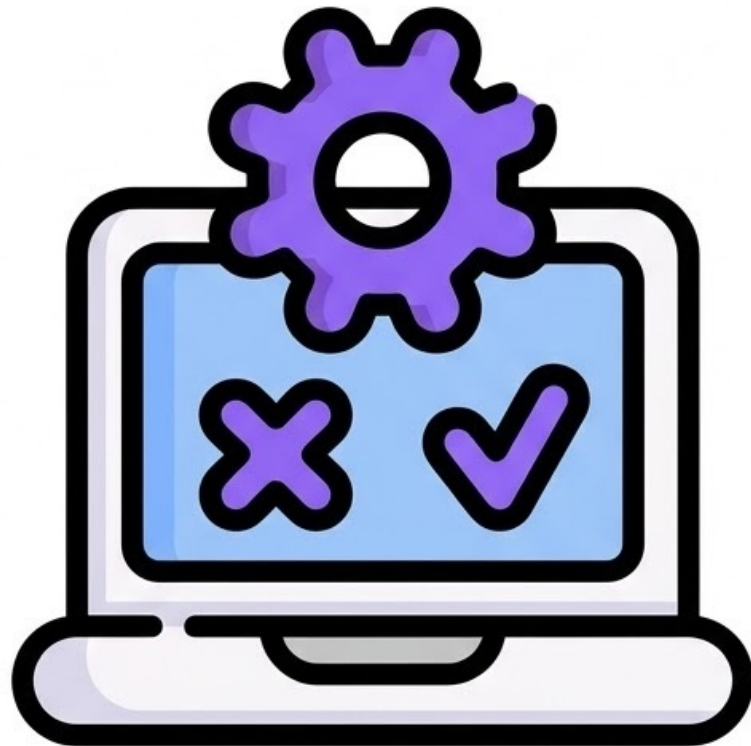
통합 테스트(Integration Test)



- 모듈 간 상호작용 검증
- 데이터 흐름 및 인터페이스 확인
- 종속성 및 외부 시스템 통합 확인

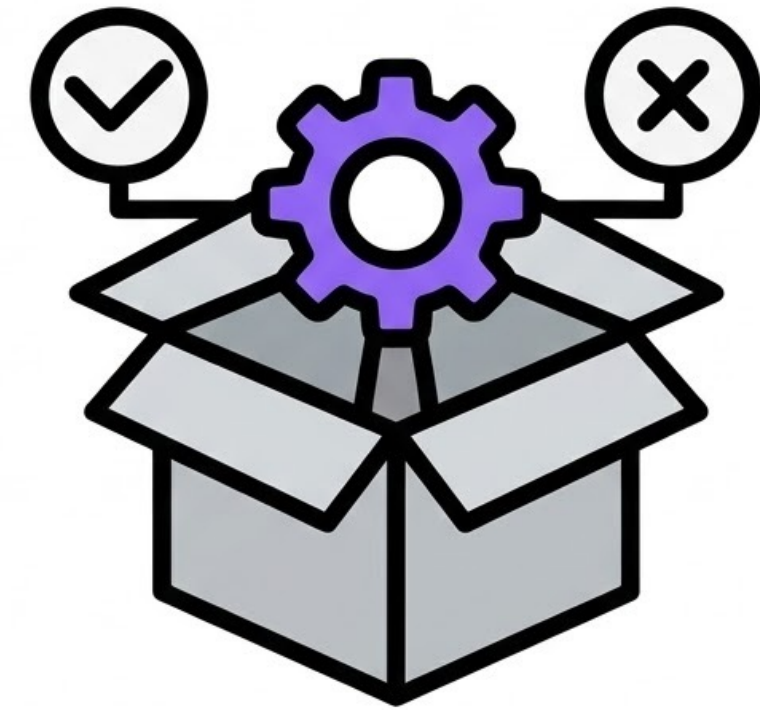


시스템 테스트(System Test)



- End-to-end 전체 시스템 기능 검증
- 실제 운영 환경과 유사한 조건
- 성능·부하·스트레스 테스트 포함

인수 테스트(Acceptance Test)



- 최종 사용자 관점의 검증
- 비즈니스 요구사항 충족 확인
- 알파·베타 테스트 및 인수 기준 검증

왜 테스트 코드를 작성해야 할까?



문제점 조기 발견

프로그램을 작은 단위로
쪼개어 검사
오류 발생 시 버그의 원인을
즉각적으로 추적 가능



리팩토링 용이성

회귀 테스트*를 통해 코드 변경
후에도 기존 기능의 정상 작동을 확신
과감하고 안전한 코드 개선 가능



개발 효율성의 극대화

초기 작성 비용은 존재하나, 교차점 이후
디버깅 시간을 압도적으로 단축하여 전체
개발 기간 감축

유닛 테스트(Unit Test)란 무엇인가?



소스 코드의 특정 모듈이 의도대로 정확히 작동하는지 격리하여 검증하는 필수 절차

유닛테스트의 F.I.R.S.T 원칙을 알아보자

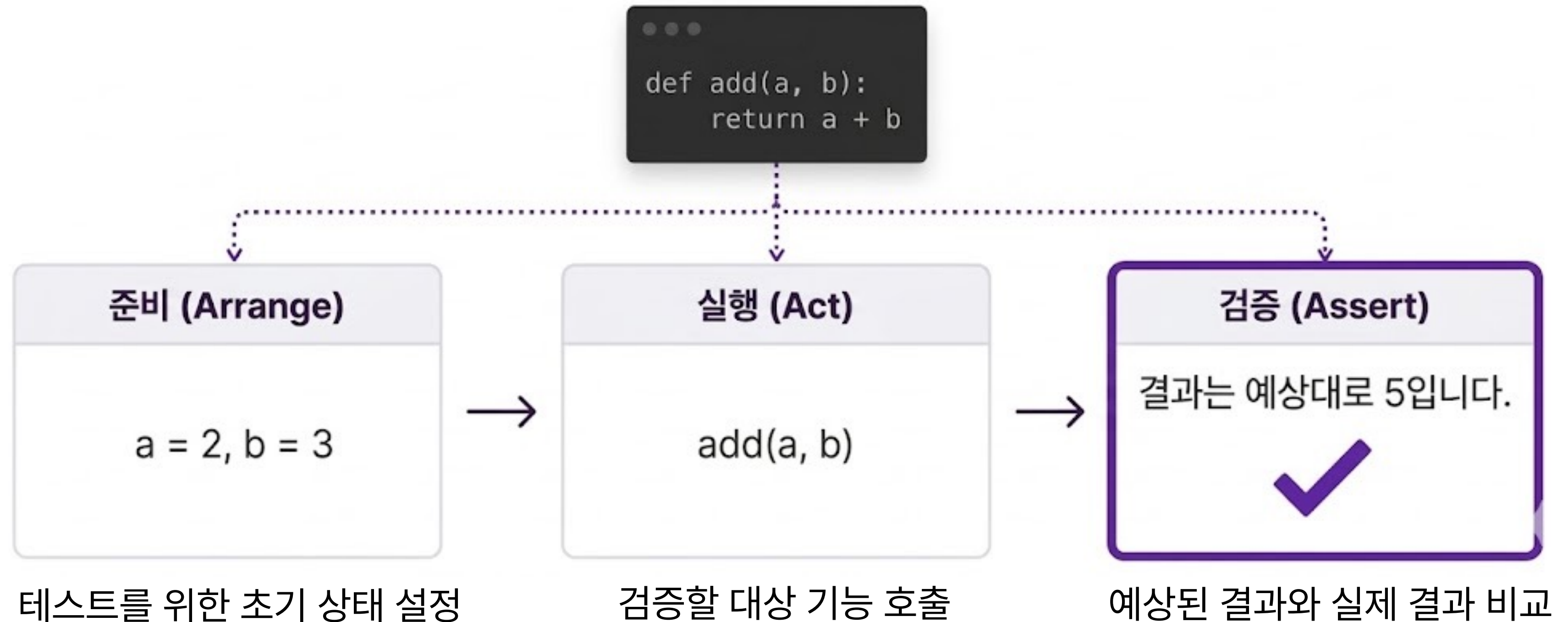
❖ 로버트 C. 마틴의 저서 클린코드 (Clean Code) 에서 소개된 유명한 개념이다.

원칙	의미	무엇을 (What)	왜 (Why)	어떻게 (How)
Fast	빠르게	수천 개 테스트를 수 초 내 실행	느리면 개발자가 안 돌림	외부 의존성(DB, 네트워크) 제거
Independent	독립적	테스트 간 순서·상태 공유 없음	실패 원인 즉시 특정	setUp/tearDown으로 격리
Repeatable	반복 가능	언제, 어디서 실행해도 동일 결과	Flaky 테스트 방지	랜덤값·시간·환경 의존 제거
Self-validating	자가 검증	성공/실패를 코드가 판단	수동 확인 제거	assertion으로 명시적 검증
Timely	적시에	프로덕션 코드와 함께 작성	테스트 가능한 설계 유도	TDD 또는 코드 직후 작성

수동 확인에서 자동 검증으로

구분	수동 확인	자동화된 테스트
검증 방식	개발자가 직접 눈으로 결과 확인	코드를 통해 정확한 결과 자동 검증
실행 과정	매번 직접 값을 입력하고 실행	명령어 한 번으로 전체 기능 일괄 실행
핵심 역할	"현재" 작성한 코드의 동작만 확인	미래의 "변경"에 대한 안전장치 (Safety Net)

테스트의 3단계 핵심 구조(AAA)



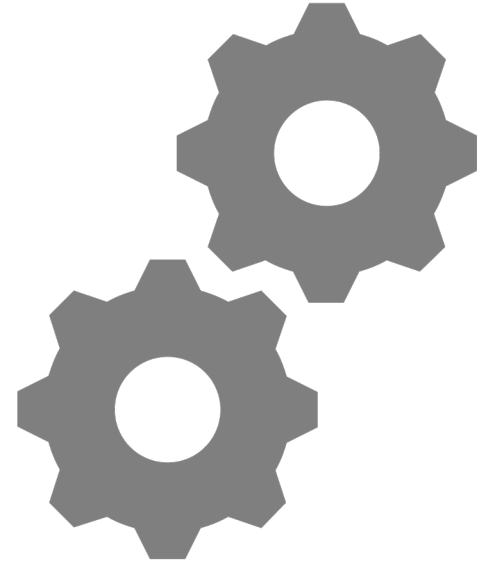
Python 테스트 구현 방법



프레임워크 비교 : unittest vs. pytest

구분	unittest	pytest
구분 및 문법	객체지향적, 무거운 assertEquals 사용, 클래스 상속 필수	함수형 작성 가능, 간결한 표준 assert 문법 사용
설치 및 편의성	파이썬 내장 라이브러리, 설치 불필요	추가 설치 필요하지만 압도적인 플러그인 생태계 제공
호환성	독립적 실행	unittest 기반 코드도 완벽히 실행 및 포용 가능

Pytest는 현대 파이썬 커뮤니티에서 가장 권장되는 유연하고 강력한 도구



기존 방식 (unittest)

```
self.assertEqual(a, b)  
self.assertIn(a, b)
```

프레임워크 전용 검증 메서드를
따로 암기해야 함



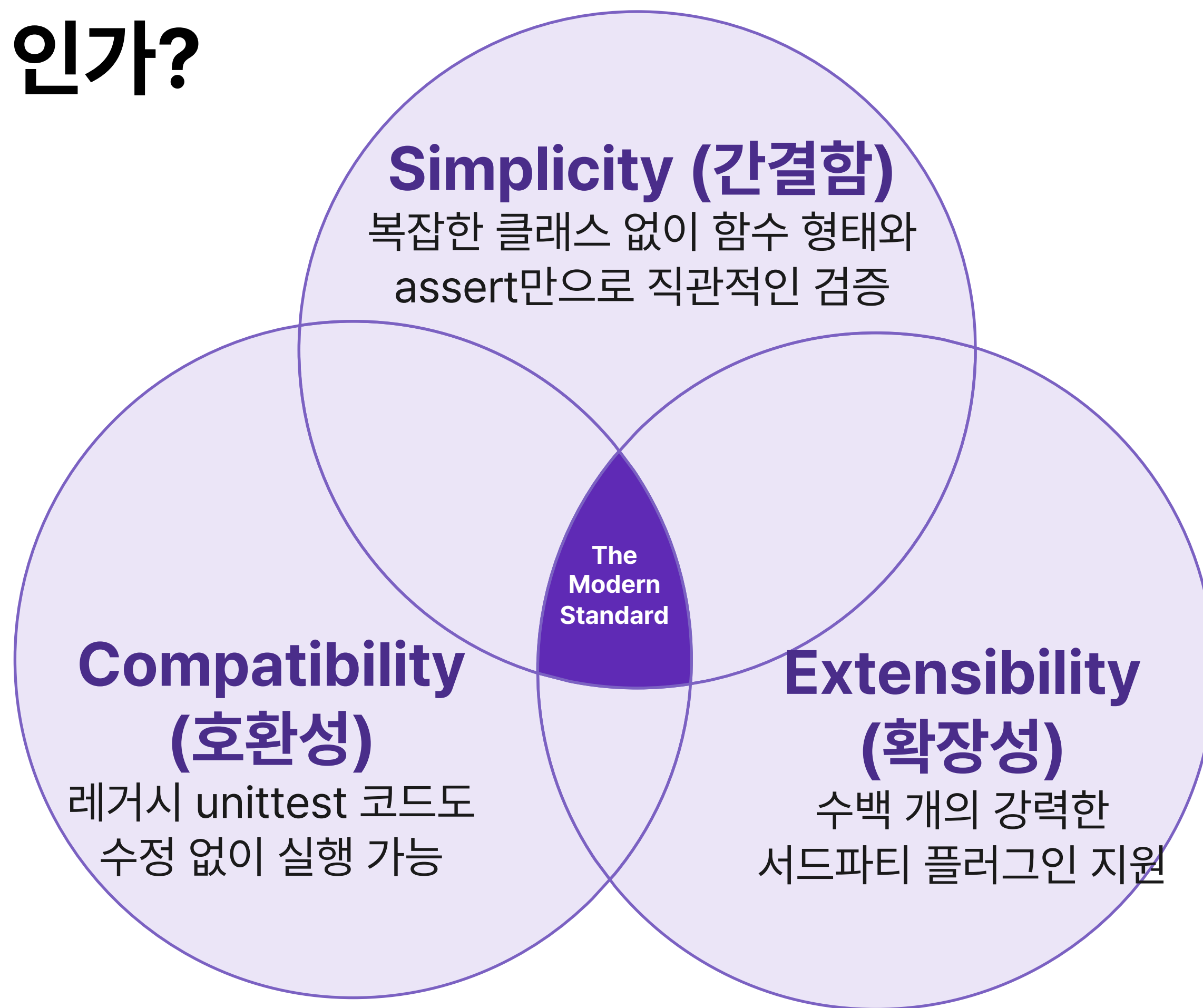
pytest 방식

```
assert a == b  
assert a in b
```

파이썬 표준 assert 문 하나로
모든 비교 연산(==, !=, <, in) 가능



왜 pytest 인가?





`pytest` vs `unittest`

실제 코드로 살펴보자. 둘 모두 똑같은 테스트를 하지만 코드양이 다르다.

unittest 방식 (상대적으로 복잡함)

```
import unittest

class TestMath(unittest.TestCase):
    def test_add(self):
        result = 2 + 3
        self.assertEqual(result, 5)

if __name__ == '__main__':
    unittest.main()
```

→ 총 8줄!

→ class, self, 특별한 메서드 등 복잡하다

pytest 방식 (상대적으로 간단함)

```
def test_add():
    result = 2 + 3
    assert result == 5
```

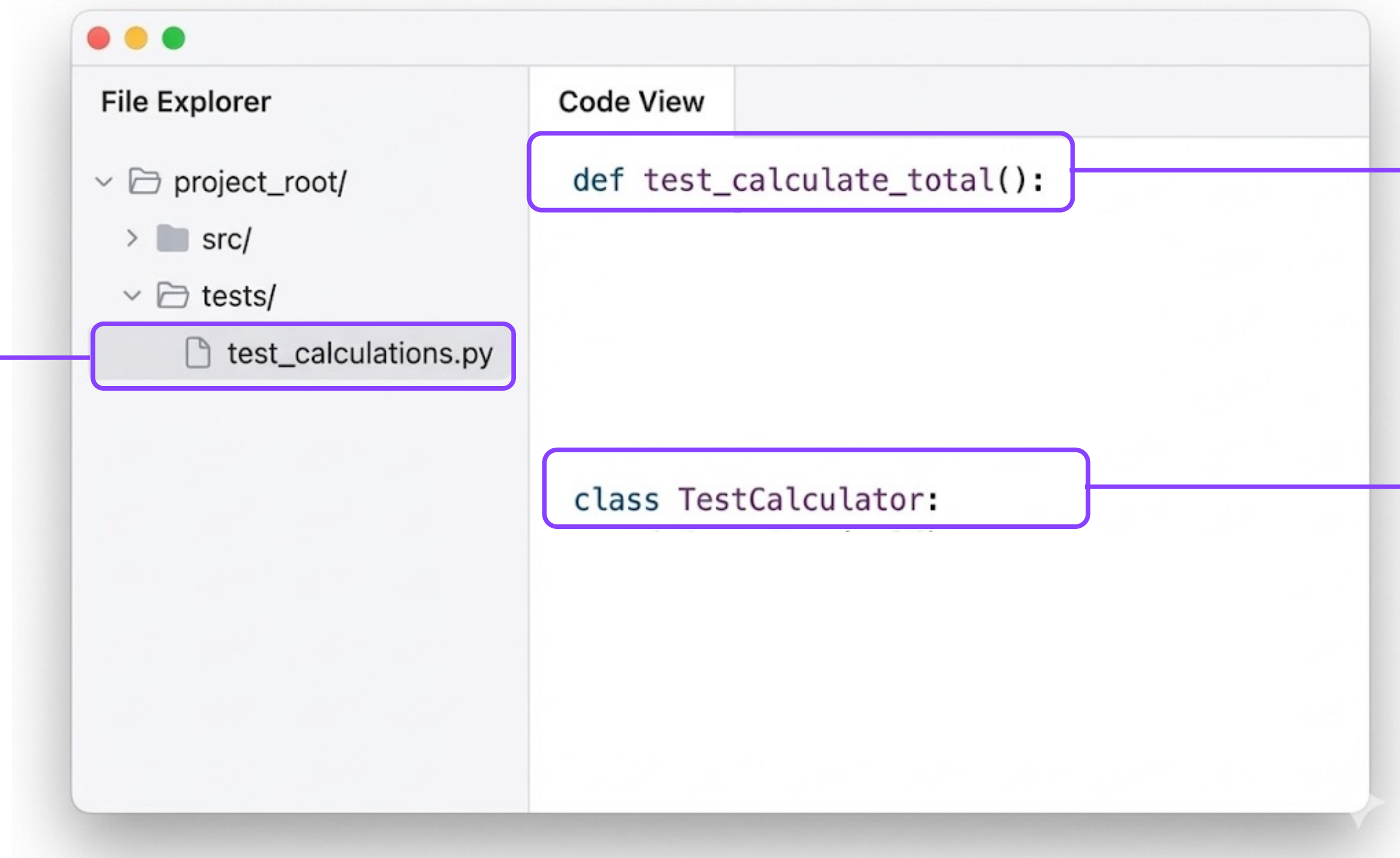
→ 총 3줄!

→ 그냥 함수와 assert만 있으면 된다.

pytest의 명칭 규약 (Naming Convention)

pytest가 테스트를 자동으로 수집(Discovery)하기 위한 필수 조건

test_*.py 또는
*_test.py
(파일명 규칙)

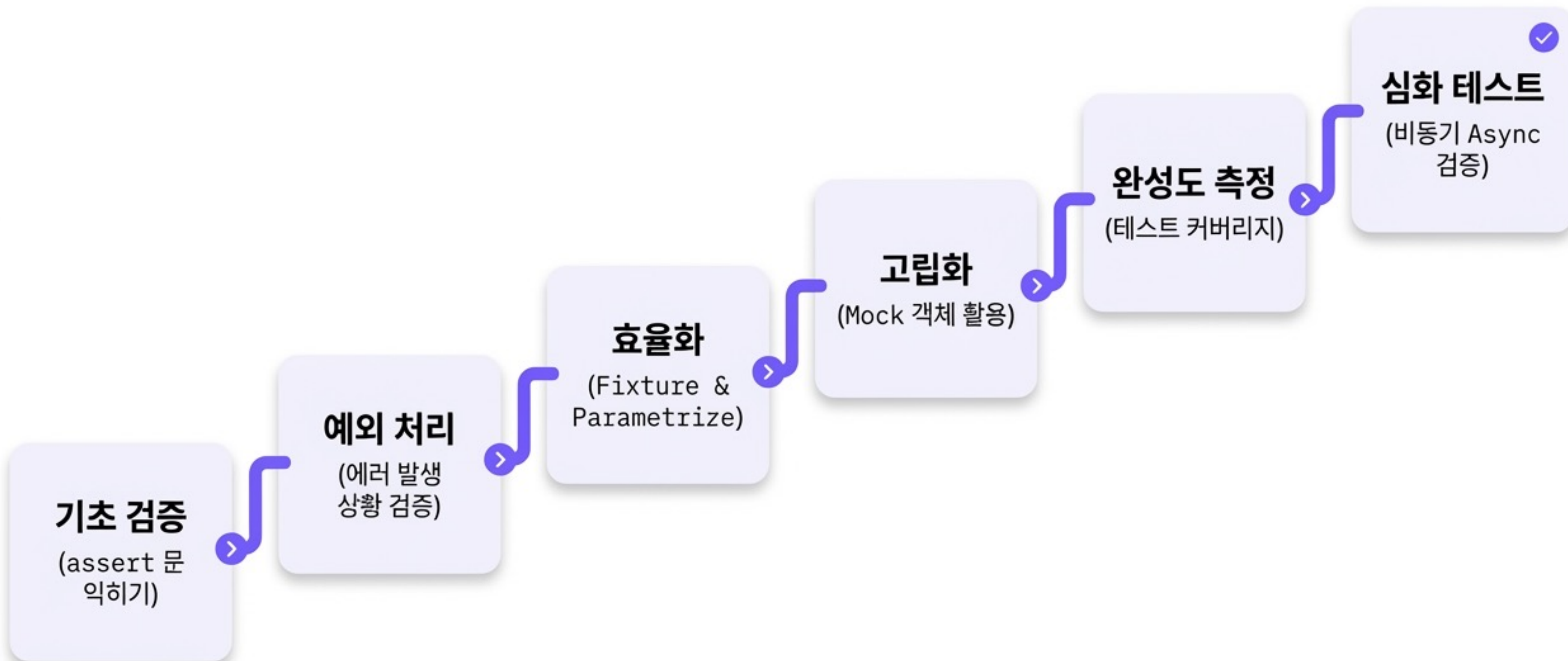


함수명은 반드시
test_로 시작

클래스로 묶을 경우
Test로 시작,
상속은 불필요



테스트 6단계 모델





pytest 에서의 assert

일반 Python의 assert는 메시지가 단순하지만, pytest는 실패 원인을 상세히 보여줌

- 기본 형태

```
assert 조건식  
assert 조건식, "실패 시 메시지"
```

- 동작 원리를 살펴보자

```
# 조건이 True → 아무 일도 안 일어남 (통과)  
assert 1 + 1 == 2  
  
# 조건이 False → AssertionError 발생 (실패)  
assert 1 + 1 == 3 # AssertionError!  
  
# 메시지 포함  
assert 1 + 1 == 3, "1+1은 3이 아닙니다"  
# AssertionError: 1+1은 3이 아닙니다
```



1단계 : 복잡한 메서드 대신 assert

pytest는 파이썬 표준 비교 연산자(==, !=, <, in)를 그대로 사용

동등성 비교

```
assert result == expected
assert result != wrong_value
```

포함 여부

```
assert "hello" in "hello world"
assert 3 in [1, 2, 3, 4, 5]
assert "error" not in log_message
```

동일성 (같은 객체인지)

```
assert result is None
assert result is not None
```

참/거짓

```
assert is_valid      # True인지
assert not is_empty  # False인지
```

비교 연산

```
assert len(items) > 0
assert price >= 0
assert age < 150
```

타입 확인

```
assert isinstance(result, list)
assert type(result) == dict
```

테스트 실행 및 결과 해석

상세 출력(verbose) 및 특정 경로 실행
-k(이름 필터), -x(첫 실패 시 중단),
-s(print 보기), -v(상세), --lf/--ff(실패만 재실행)

```
$ pytest -v tests/  
tests/test_app.py .F [100%]  
  
===== FAILURES =====  
Traceback (most recent call last):  
AssertionError: assert 404 == 200
```

→ 전체 진행률

← 실패 상세 이유 및 오류 위치 지칭

← . (Pass) 성공, F (Fail) 실패



pytest 실행 흐름

